TP 4 Booking : Classes abstraites – Surcharge – Polymorphisme -interfaces – Collections

⚠ Ce TP4 Booking est à faire dans le même binôme que les TP2 et TP3. Ce TP4 sera à déposer ultérieurement sur BoostCamp, avec le TP2, selon les consignes qui vous seront mentionnées.

Exercice 1 : définition et implémentation d'une classe abstraite et d'une classe qui en hérite

- Pour ce TP4, qui fait suite au TP3 du Diagramme de classes et concepts de base précédent, imaginez un diagramme de classes avec une classe abstraite à partie des classes précédentes, en justifiant votre choix judicieusement à partir de votre TP2 de cas d'utilisation et Diagramme de classes et votre TP3 de concepts de base en y ajoutant les propriétés suivantes :

- Prévoir au moins une classe abstraite avec au moins une méthode abstraite, en justifiant votre choix.
- Les attributs nécessaires, dont certains **private** si besoin avec des getters ou setters, d'autres **public** ou **protected** en cad d'héritage entre classes.
- Les constructeurs nécessaires qui initialisent les attributs, compris entre constructeur savec des paramètres hérités avec **super**.
- Des méthodes « getter » qui retournent si besoin des attributs **private**, ou éventuellement **protected** pour des classes en dehors de celles héritées.
- Des méthodes *afficher*() pour afficher les attributs de chacune des classe.

Pourquoi la méthode est-elle abstraite dans la classe abstraite ? Pourriez-vous définir votre classe abstraite comme interface ? Pourquoi ? En quoi peut-on considérer qu'il y a du polymorphisme dans les méthodes redéfinies ?

- Dans ce TP Booking, définissez et implémentez une classe qui hérite d'une la classe abstraite avec les propriétés nécessaires suivantes :

- Au moins 2 attributs **private**.
- Les 3 constructeurs suivants qui initialisent les attributs(s) par héritage de la super-classe :
  1) Le premier constructeur par défaut (sans paramètre) qui initialise les attributs avec des valeurs par défaut.
  2) Le second constructeur avec les paramètres associés aux attributs.
  3) Le troisième et dernier constructeur par copie avec en paramètre un objet de la classe et qui copie les attributs de cet objet dans les attributs de l'objet autoréférencé **this**.
- Des getters qui retournent chacun la valeur d'un attribut **private**.

Pourquoi faut-il utiliser le mot **this** (autoréférence à un objet) dans ce troisième et dernier constructeur par copie ?

- Une méthode *afficher*() qui affiche les attributs de la classe héritée.
- D'autres méthodes nécessaires de la classe héritée.

Comment réagit le compilateur si la classe héritant de la classe abstraite n'implémente pas la méthode abstraite ? Pourquoi ?

Exercice 2 : introduction de la méthode *toString()*

Un objet contient par défaut plusieurs méthodes de base dont la méthode redéfinie ***toString()*** héritée de la classe générale Object. Réécrivez toujours la méthode ***toString()*** ne serait-ce que pour déjà faciliter le débogage

Exemple :
@Override // surcharge d'une méthode héritée et redéfinie
public String toString()
{ return … ; // Une description synthétique de votre texte **}**

- En vous inspirant de l'exemple ci-dessus de la méthode standard ***toString()*** (redéfinie par héritage de la classe Object), implémentez cette méthode dans votre classe abstraite et votre classe qui en hérite.

Exercice 3 : implémentation du main dans une autre classe

- Dans une classe **main** d'une autre classe, effectuer les instructions suivantes :
  - Instancier un premier objet de la classe qui hérite de la classe abstraite avec le premier constructeur par défaut (voir **exercice 1**).
  - Lire au clavier au moins un variable définie en vérifiant par une boucle que sa valeur est correcte.

    ⚠ Pour ceux qui connaissent les exceptions, servez-vous de ces exceptions prédéfinies pour vérifier la valeur d'une variable. Pour plus d'informations sur les exceptions vues plus tard, consulter les support Exceptions et Illustration des exceptions sur la page BoostCamp Section : Semaine 5 16/03/2026 : Exceptions - Fichier texte - Sérialisation | POO Java ING3 2026 | BOOSTCAMP.
  - Instancier un second objet de la classe qui hérite de la classe abstraite avec le second constructeur (voir **exercice 1**).
  - Instancier un troisième objet de la classe qui hérite de la classe abstraite avec le troisième constructeur par copie (voir **exercice 1**).
  - Pour les 3 objets précédents, effectuer les instructions suivantes :
    - Appeler la méthode *afficher*() de l'**exercice 1**)
    - Afficher leurs informations sous forme de String en appelant la méthode ***toString()*** de **l'exercice 2**.

Exercice 4 : suite de l'exercice 1 avec une collection d'objets

1) Dans le diagramme de classes et les classes de l'**exercice 1**, intégrer une interface constituée d'une liste d'objets de la classe abstraite définie dans l'**exercice 1**. Cette nouvelle classe dispose de la méthode *ajouter* pour ajouter un objet de cette classe dans la liste des objets de cette classe.

Dans ce diagramme, la classe abstraite implémente l'interface **java.lang.Comparable<***nom de la classe abstraite***>** et redéfinit la méthode *compareTo* de cette interface pour comparer l'un des critères, comme le prix. Voir des exemples dans les supports cours Interfaces comparable et comparator et Sources des exemples du cours Interfaces Comparable et Comparator de la page BoostCamp Section : Semaine 4 09/03/2026 : Polymorphisme - Classes abstraite - Interface - Collections | POO Java ING3 2026 | BOOSTCAMP.

2) Implémenter en Java l'interface ci-dessus avec les propriétés suivantes :
- Un attribut pour la liste dynamique d'objets de cette interface : **ArrayList**<*nom de la classe*> à instancier
- La méthode *ajouter* ci-dessus

Pour l'utilisation de la classe **ArrayList** (collection dynamique), voir le tutoriel Exemples avec la classe ArrayList et toutes les caractéristiques de la classe **ArrayList** sur le site suivant : **https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html**

Quels sont les avantages d'une **ArrayList** par rapport à un tableau ?

3) Dans une classe qui implémente l'interface **java.lang.Comparable<***Forme***>**, implémenter la méthode redéfinie *compareTo* pour comparer la velur de 2 critères (par exemple le prix) entre 2 objets de la classe, avec une méthode.

Pour vous aider, servez-vous des 2 liens suivants : **cours Interfaces comparable et comparator** et de l'API Comparable (Java Platform SE 8)

4) Compléter le **main** de la classe *TestExercice1* avec les instructions suivantes :
- Instancier un objet de la classe du **1)** de l'**exercice 4.**
- Dans une boucle, instancier et ajouter chaque objet de la classe de l'**exercice 4** dans la liste d'objets d'objets précédente, avec la méthode nécessaire.
- Dans cette boucle, de préférence faire un menu avec plusieurs options.

Pour connaitre le type de chaque forme de la liste, utilisez le casting approprié en vous inspirant des liens suivants : Aide : Java instanceof - javatpoint et casting - Downcasting in Java - Stack Overflow
Pour trier votre liste avec la méthode **java.util.Collections.sort**, voir l'API **https://docs.oracle.com/javase/8/docs/api/java/util/Collections.html** et le lien Comment comparer puis trier une liste (ArrayList ou List) d'objets en Java en utilisant les interfaces Comparable et Comparator et la classe Collections ?

Exercice 5 : pour aller plus loin

Pour aller plus loin : Définissez des filtres sur différents critères de sélection, comme le lieu, le type d'hébergement, la date etc.

Vous pouvez vous aider de chatGPT, Copilot ou équivalent pour des conseils, mais sans génération de code sans réflexion ni compréhension de votre part.

N'oubliez pas de citer vos sources : auteurs, liens web, etc.